

# Lösung von SeaShanty mit tabularem Reinforcement Learning

Task 1 – Reinforcement Learning (Prof. Thomas Nierhoff, OTH Amberg-Weiden)

Autor: Himanshu Mohan Nikam

## 1. Projektüberblick

Ziel von Task 1 ist es, die Umgebung `SeaShanty(size=12, period=6)` mit *möglichst wenigen Episoden* zu lösen. Der Agent bewegt sich auf einem  $12 \times 12$ -Gitter, in dem sich tödliche „Wasser“-Zellen in jedem Schritt nach einem festen Muster bewegen. Der Agent bekommt +100 für das Ziel, −100 für Wasser und 0 sonst. Da sich das Wasser mit dem Schritt-Zähler  $t$  bewegt und `reset()`  $t$  zu Beginn jeder Episode auf 0 setzt, ist dieselbe Zelle mal sicher und mal tödlich.

Das Projekt besteht aus mehreren eigenen Dateien. Die vorgegebenen Dateien (`gridworld.py`, `plot.py`, `config.py`) wurden nicht verändert:

- `task1.py` – Q-Learning und Double Q-Learning, plus die geforderte Lernkurve und die Q-/V-Tabellen-Plots.
- `sarsa.py` – SARSA und n-step SARSA.
- `tune.py` – Optuna-Tuning für die Methoden in `task1.py`.
- `sarsa_tune.py` – Optuna-Tuning für die Methoden in `sarsa.py`.

Gemessen wird die „reale“ kumulierte Belohnung pro Episode (die Summe der echten, *nicht* geschapten Belohnungen), gemittelt über 100 verschiedene Umgebungen (Seeds 1000–1099). Für das Tuning werden andere Seeds (0–29) benutzt als für die finale Auswertung.

## 2. Ansätze

### 2.1 Wertbasiert: Q-Learning (in `task1.py`)

Q-Learning lernt direkt die optimale Aktions-Bewertung  $Q(s, a)$ . Zusätzlich haben wir **Double Q-Learning** umgesetzt: Es benutzt zwei Tabellen und trennt die Auswahl der Aktion von ihrer Bewertung, um die Überschätzung der Werte zu reduzieren – das ist neben der großen −100-Strafe besonders wichtig.

### 2.2 On-policy: SARSA (in `sarsa.py`)

SARSA bewertet die Politik, der es wirklich folgt, und ist dadurch *vorsichtiger* nahe am tödlichen Wasser. Wir haben **SARSA** und **n-step SARSA** ( $n = 4$ ) umgesetzt; n-step gibt die Belohnungs-Information mehrere Schritte weiter zurück und verteilt so das Lob schneller entlang des Weges zum Ziel.

## 3. Wichtige Techniken

- **Phase Augmentation.** Reines positionsbasiertes Q-Learning verletzt die Markov-Eigenschaft, weil sich das Wasser mit  $t$  bewegt; so löst es nur etwa 10 % der Umgebungen. Wir erweitern den Zustand um die Phase: `state = position*K + (t mod K)` mit  $K = 38 \approx \text{round}(2\pi \cdot \text{period})$ . Da jeder Lösungsweg kürzer als  $K$  Schritte ist, gilt  $t \bmod K = t$ , und der Zustand wird wieder Markov.
- **Potential-based Reward Shaping.** Bei Schrittkosten 0 ist die Belohnung sehr dünn. Wir addieren  $F = \gamma \Phi(s') - \Phi(s)$  (Ng et al., 1999) mit  $\Phi = -(\text{Manhattan-Distanz zum Ziel})$ , normiert

und mit einem Gewicht `SHAPE_W` skaliert. Das ist policy-invariant, verändert also die optimale Politik nicht, sondern lenkt nur das Lernen. Die reale Belohnung enthält  $F$  nie.

- **Optimistische Initialisierung.** Ein optimistischer Q-Startwert bringt die Greedy-Politik dazu, systematisch neue Aktionen zu probieren – *ohne* die Zufalls-Tode, die eine hohe Epsilon-Schwelle in dieser tödlichen Umgebung verursachen würde.

## 4. Versuchsaufbau

Alle Agenten sind einfache NumPy-Tabellen. Die Hyperparameter ( $\gamma$ ,  $\alpha$ , Epsilon-Start, Epsilon-Decay, optimistischer Startwert, Shaping-Gewicht und  $n$  für n-step SARSA) werden mit **Optuna** gesucht (50 Trials pro Methode). Die Tuning-Seeds (0–29) sind von den Auswertungs-Seeds (1000–1099) getrennt. Das Tuning-Ziel maximiert die mittlere reale Belohnung über alle Episoden und belohnt so *schnelles* Lernen.

## 5. Ergebnisse

Abbildung 1 zeigt die geforderte Lernkurve für Q-Learning: die reale Belohnung pro Episode, gemittelt über 100 Umgebungen. Nach einer Anfangsphase mit vielen frühen Toden steigt die Kurve und nähert sich +100, sobald die Greedy-Politik das Ziel zuverlässig erreicht. Double Q-Learning verhält sich ähnlich, mit etwas ruhigeren Werten.

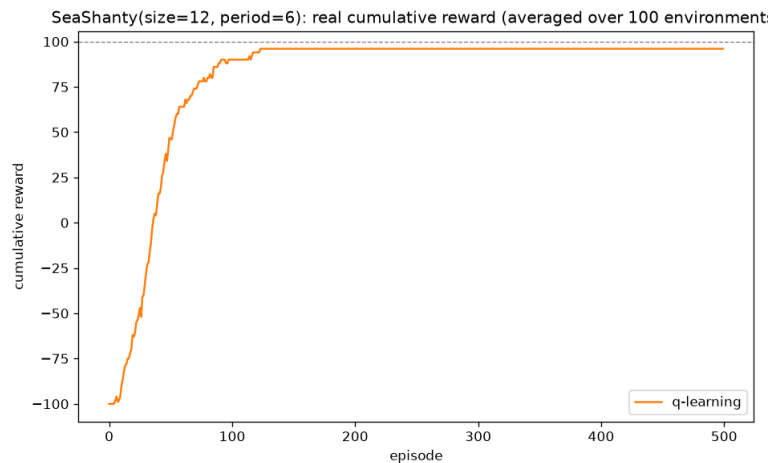


Abbildung 1: Q-Learning: reale kumulierte Belohnung pro Episode, gemittelt über 100 Umgebungen (Seeds 1000–1099).

Abbildung 2 vergleicht alle vier Methoden auf denselben 100 Umgebungen; jede Methode benutzt dabei ihre *eigenen* mit Optuna getunten Hyperparameter. Q-Learning und 1-Schritt-SARSA steigen zügig auf etwa +95 (gelöst nach rund 100–120 Episoden). Double Q-Learning und n-step SARSA lernen deutlich langsamer: Ihre Kurven steigen stetig, erreichen innerhalb von 300 Episoden aber nur die „Überlebens“-Politik um 0 und noch nicht das Ziel (Gründe siehe Diskussion).

## 6. Fehlgeschlagene Versuche

Wir berichten bewusst auch, was *nicht* funktioniert hat.

### 6.1 Reward Shaping ohne Skalierung

Ohne die Skalierung mit `SHAPE_W` bleibt das Potential im Bereich  $[0, 1]$ ; die Steigung ist dann nur etwa  $1/22 \approx 0,045$  pro Schritt und damit viel zu klein gegenüber den  $\pm 100$  der Endzustände. Das haben wir für beide Familien getestet:

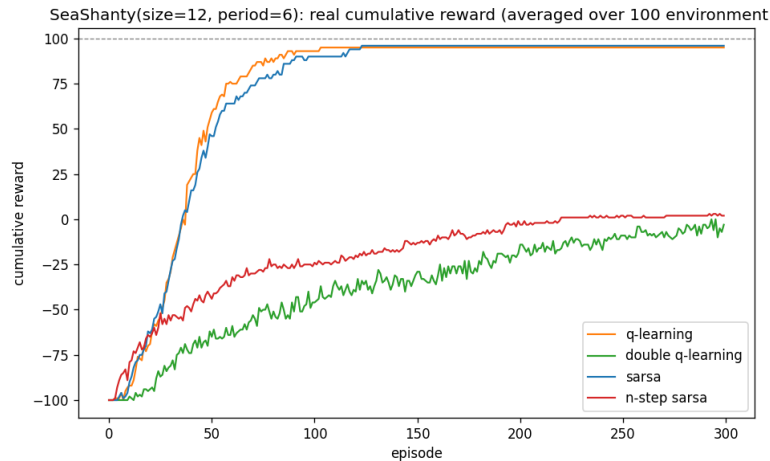


Abbildung 2: Vergleich aller vier Methoden: reale kumulierte Belohnung pro Episode, gemittelt über 100 Umgebungen (Seeds 1000–1099).

| Methode                | Verhalten    | Reale Belohnung (Ep 300) | Anmerkung                            |
|------------------------|--------------|--------------------------|--------------------------------------|
| Q-Learning             | löst schnell | → +95                    | primäre Lösung, am schnellsten       |
| SARSA (1-Schritt)      | löst schnell | → +95                    | fast so schnell wie Q-Learning       |
| Double Q-Learning      | überlebt     | ≈ 0                      | konvergiert langsam (zwei Tabellen)  |
| n-step SARSA ( $n=4$ ) | überlebt     | ≈ 0                      | konvergiert langsam (n-step>Returns) |

Tabelle 1: Vergleich der Methoden auf den Auswertungs-Umgebungen (Seeds 1000–1099) nach 300 Episoden, jede mit ihren *eigenen* getunten Hyperparametern. Q-Learning und 1-Schritt-SARSA lösen; Double Q-Learning und n-step SARSA erreichen nur die Überlebens-Politik ( $\approx 0$ ).

- **SARSA / n-step SARSA:** Die Kurve bleibt bei etwa 0 stehen – der Agent „überlebt“ nur und meidet das Wasser – und erreicht das Ziel nur selten; rund 10 % der Umgebungen werden gelöst.
- **Q-Learning / Double Q-Learning** (auf einem anderen Rechner getunt, siehe `qlearning_without_scaling`). Das Ergebnis ist stark verrauscht – etwa 42 % der Umgebungen gelöst, aber viele Tode. Die gemittelte Kurve steht bei Episode 199 noch bei  $-10$ . Selbst nach 50 Optuna-Trials war der beste Tuning-Wert nur  $-96,7$  (Q-Learning) bzw.  $-3,4$  (Double Q-Learning) – ein deutliches Zeichen, dass fast nichts gelernt wurde.

Mit Skalierung ( $\text{SHAPE\_W} \approx 80\text{--}125$ ) erreicht Q-Learning das Ziel dagegen zuverlässig, und die Kurve steigt schon nach etwa 125–150 Episoden auf  $+100$ . Das Skalieren des Potentials ist policy-invariant (nur die Lerngeschwindigkeit ändert sich, nicht die optimale Politik) und war hier der entscheidende Schritt.

## 6.2 Weitere gescheiterte Ideen

- **Hohe Epsilon-Schwelle zur Exploration:** fatal. Zufällige Wege sterben im tödlichen Wasser, bevor sie das Ziel finden. Die Exploration muss stattdessen von der geschapten, optimistischen Greedy-Politik kommen.
- **Nur Phase Augmentation ohne Shaping:** löst die Umgebung zwar, braucht aber über 4000 Episoden.
- **Tuning auf schwachem (un-skaliertem) Shaping:** liefert fast kein Signal (fast alle Trials  $\approx -99$ ), also praktisch zufällige Hyperparameter. Werden diese später mit skaliertem Shaping benutzt, sind die Ergebnisse schlechter als beim Original. Das Tuning sollte immer mit dem

*skalierten* Shaping laufen.

## 7. Diskussion

Die größte Schwierigkeit ist die Nicht-Stationarität der Umgebung, nicht die Lernregel. Mit Phase Augmentation wird das Problem Markov, und Off-policy-Q-Learning löst es zuverlässig; ohne Augmentation hilft kein Tuning.

Interessant ist, dass mit *skaliertem* Reward Shaping und getunten Hyperparametern nicht nur das off-policy Q-Learning, sondern auch das on-policy 1-Schritt-SARSA die Aufgabe zuverlässig und etwa gleich schnell löst (Abbildung 2). Das starke Shaping-Signal überwiegt hier die sonst typische Vorsicht von SARSA. Ohne Skalierung (Abschnitt 6) bleibt SARSA dagegen bei der reinen „Überlebens“-Politik bei Belohnung 0 stehen – entscheidend ist also das Shaping, nicht die Wahl von on- gegen off-policy.

Die beiden langsameren Methoden – **Double Q-Learning** und **n-step SARSA** – wurden hier bewusst mit ihren *eigenen* getunten Hyperparametern ausgewertet und bleiben trotzdem innerhalb von 300 Episoden bei der Überlebens-Politik (Belohnung um 0). Ihre zusätzliche Maschinerie bremst das Lernen in dieser Aufgabe eher, als dass sie hilft: Double Q-Learning verteilt die Updates auf zwei Tabellen und halbiert so praktisch die effektive Lernrate pro Tabelle, und die n-step>Returns machen die Updates verrauschter. Beide Kurven steigen jedoch stetig weiter und würden mit mehr Episoden voraussichtlich ebenfalls das Ziel erreichen. Bemerkenswert ist, dass die Trennung nicht entlang on-/off-policy verläuft – je eine Methode aus beiden Lagern löst, je eine überlebt nur –, sondern entlang der Komplexität der Lernregel.

## 8. Fazit

Wir haben zwei Familien tabularer Agenten verglichen – Q-Learning/Double Q-Learning und SARSA/n-step SARSA – zusammen mit Phase Augmentation, skaliertem Reward Shaping und optimistischer Initialisierung. Phase-augmentiertes Q-Learning ist der schnellste und zuverlässigste Ansatz und bringt die mittlere reale Belohnung in etwa 100–120 Episoden auf +95; mit skaliertem Shaping löst auch 1-Schritt-SARSA die Aufgabe ähnlich schnell, während Double Q-Learning und n-step SARSA – selbst mit eigenem Tuning – innerhalb von 300 Episoden nur die Überlebens-Politik erreichen und mehr Episoden benötigen. Ohne die Skalierung des Shaping-Potentials scheitern dagegen alle Methoden, wie die dokumentierten Versuche zeigen.

## 9. Quellen

### Literatur

- (Anthropic), Claude (2026). *[Your Chat Title/Topic Here]*. <https://claude.ai/share/2359a783-c33a-430a-8f6e-b91dc0d88510>. Accessed: July 18, 2026.
- Miller, Tim (2023). *Reward Shaping*. <https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html>.
- Professor, Thomas Nierhoff (2026). *Deep Reinforcement Learning*. Vorlesungsfolien. Modul: Reinforcement Learning.
- Zhao, Sophie (Juli 2025). “Potential-Based Reward Shaping in Reinforcement Learning”. In: *Medium*. 6 min read.